CodInClub
Powered by BridgeLabz

Category B Assignment Problems

5 problems · Medium · STEP SEM-3 · Week 4 · Constructors and Java Keywords · for independent take-home practice

How This Set Relates to the In-Class Problems

This assignment covers the exact same five ideas as the in-class Category B set, in the same order — this() chaining, the this keyword, final, static blocks, and instanceof — each one still scaffolded with a for loop, an if-else, and an array, the same way. What's different is the scenario behind every problem: hackathon registration instead of a library, canteen inventory instead of payroll, parking fines instead of late fees, a library card instead of a student ID, and canteen payments instead of hostel fees. If a problem here feels difficult, the in-class version of the same concept is the right place to go back and re-check your understanding before attempting it again.

A1. Overloaded Constructors for Hackathon Registration

Scenario

A hackathon's registration desk stays open all evening. Some participants arrive already part of a team and give their team name at signup; others register solo and get a placeholder team name until they're matched with one later that night.

> **BASIC CONSTRUCT BEING REINFORCED**
>
> *A for loop over two parallel arrays (names and teamNames, where some teamNames entries are blank) with an if-else inside to decide which constructor fits each participant — the exact same shape as the library book problem, with a different scenario.*

Task

- *Define a class Participant with fields String name, String teamName, and boolean registered.*
- *Write Participant(String name, String teamName) that sets all three fields directly, with registered set to true.*
- *Write a second constructor Participant(String name) for solo entries, which uses this(name, "Unassigned") to chain to the constructor above.*
- *Add printStatus() that prints name, teamName, and registered on one line.*
- *You're given String[] names and String[] teamNames (some teamNames entries are an empty string ""). Using a for loop, check with if-else whether teamNames[i] is empty, and call the matching constructor for that participant.*
- *Call printStatus() for every participant inside the same loop.*

Suggested Method Signature(s)

```
public Participant(String name, String teamName)
public Participant(String name) // chains via this(...)
```

Sample Input / Output

Input	Output

names = {"Ravi", "Meera", "Karthik", "Divya"} *teamNames = {"ByteBusters", "", "CodeCrafters", ""}*	*Ravi | ByteBusters | Registered: true* *Meera | Unassigned | Registered: true* *Karthik | CodeCrafters | Registered: true* *Divya | Unassigned | Registered: true*

Concepts covered: *for loop batch processing, if-else decision-making, arrays, constructor overloading, this() chaining.*

A2. this Keyword for Canteen Inventory — Batch Restock

Scenario

Every Friday evening, the campus canteen restocks the same fixed quantity of every item on the shelf at once, ahead of the weekend rush.

> ***BASIC CONSTRUCT BEING REINFORCED***
> *A for loop over an array of Item objects, calling the same restock() method on every one of them with the same amount — the same batch-update shape as the payroll bonus problem.*

Task

- *Define a class Item with fields String itemName and int stock.*
- *Write a constructor Item(String itemName, int stock) where the parameter names exactly match the field names, and resolve the clash using this.itemName = itemName; and this.stock = stock;.*
- *Add restock(int stock) that adds the parameter to this.stock — the same kind of clash, resolved the same way.*
- *Create an array of 4 Item objects with different starting stock levels.*
- *Using a for loop, call restock(20) on every item in the array, then print each item's name and final stock inside the same loop.*

Suggested Method Signature(s)

```
void restock(int stock)
```

Sample Input / Output

Input	*Output*
4 items with starting stock {15, 40, 8, 25}, restock(20) applied to each	*Samosa | Final Stock: 35* *Tea Powder | Final Stock: 60* *Bread | Final Stock: 28* *Biscuit Packs | Final Stock: 45*

Concepts covered: *this keyword (constructor and method field/parameter clashes), for loop over an array of objects, basic addition logic.*

A3. final Method — Parking Overstay Fine Calculator

Scenario

Campus parking issues a fine for every vehicle that overstays its allotted free time — but the fine formula must behave identically for every ticket, with no risk of a future "VIP parking" subclass quietly charging a different rate.

> ***BASIC CONSTRUCT BEING REINFORCED***
> *A for loop over parallel arrays (tickets and their overstay minutes) with an if-else to decide whether a ticket owes a fine at all — the same skip-or-process shape as the late-fee problem.*

Task

- *Define a class ParkingTicket with fields String vehicleNo and double ratePerMinute.*
- *Add a final method calculateFine(int overstayMinutes) that returns overstayMinutes * ratePerMinute, and cannot be overridden by any subclass.*
- *Add a final method printReceipt(int overstayMinutes) that prints vehicleNo and the computed fine.*
- *You're given a String[] of 4 vehicle numbers, a matching double[] of rates, and a matching int[] of overstayMinutes values (some zero or negative).*
- *Using a for loop with an if-else check, call printReceipt() only for tickets where overstayMinutes > 0; for every other ticket, print "<vehicleNo> - No fine, within allotted time" instead.*

Suggested Method Signature(s)

```
final double calculateFine(int overstayMinutes)
final void printReceipt(int overstayMinutes)
```

Sample Input / Output

Input	*Output*
vehicleNos = {"TN09AB1234","TN22CD5678","TN09EF9012","TN10GH3456"} *ratePerMinute = {2, 2, 3, 2}* *ovestayMinutes = {15, 0, -5, 8}*	*TN09AB1234 - Fine: Rs 30.0* *TN22CD5678 - No fine, within allotted time* *TN09EF9012 - No fine, within allotted time* *TN10GH3456 - Fine: Rs 16.0*

Concepts covered: *final method (locking down behaviour against overriding), for loop over parallel arrays, if-else branching / skip logic.*

A4. Static Block — Library Membership Card Setup

Scenario

At the start of every semester, the campus library issues membership cards to a fresh batch of students — and wants the library's name and the card's validity year set exactly once, no matter how many cards get printed that day.

> ***BASIC CONSTRUCT BEING REINFORCED***
>
> *A for loop over a String[] of student names to create several MembershipCard objects one after another — confirming, the same way as the student-portal problem, that the static block still fires only once no matter how many times the loop runs.*

Task

- *Define a class MembershipCard with static fields String libraryName and String validUntil, and an instance field studentName.*
- *Add a static block that sets libraryName to "SRM Central Library" and validUntil to "May 2027", printing "Library info loaded" exactly once.*
- *Add a constructor MembershipCard(String studentName) that sets the studentName field.*
- *You're given a String[] of 5 student names. Using a for loop, create a MembershipCard for each name and print a short confirmation line for each one inside the loop.*
- *Check your output: “Library info loaded” should appear exactly once, no matter how many cards the loop creates.*

Suggested Method Signature(s)

```
static { ... }
```

Sample Input / Output

Input	Output
names = {"Ananya", "Rohan", "Priya", "Arjun", "Sneha"}	Library info loaded Membership card issued: Ananya Membership card issued: Rohan Membership card issued: Priya Membership card issued: Arjun Membership card issued: Sneha

Concepts covered: static block (one-time class-loading setup), for loop object creation from an array, static vs instance fields.

A5. instanceof Inside a Loop — Canteen Closing-Time Payment Dispatch

Scenario

At closing time, the canteen POS system replays the day's transactions in one batch — some paid by cash, some by card — and card transactions carry a small processing fee that cash transactions don't.

> **BASIC CONSTRUCT BEING REINFORCED**
>
> A for loop over an array of Payment references (some of which actually point to CardPayment objects underneath), with an if-else built on instanceof inside the loop to decide whether a processing fee applies — plus a running total updated the same way you've accumulated sums in a plain array before.

Task

- Define a class Payment with a method pay(double amount) that prints the amount paid.
- Define a class CardPayment extends Payment, adding payWithProcessingFee(double amount) that adds a 2% fee to the amount before printing the total charged.
- Write processTransaction(Payment payment, double amount) that uses instanceof to check whether payment is really a CardPayment; if so, cast it and call payWithProcessingFee(), otherwise call pay().
- Build a Payment[] array of 5 references — 3 created as CardPayment, 2 created as plain Payment, stored together in the same array.
- Using a for loop, call processTransaction() for every entry with its own amount (from a matching double[] array), and keep a running double totalCollected that adds the actual amount charged (including any fee) for each transaction.
- After the loop, print totalCollected.

Suggested Method Signature(s)

```
void processTransaction(Payment payment, double amount)
```

Sample Input / Output

Input	Output
payments = {Card, Cash, Card, Cash, Card} (in that order) amounts = {100, 50, 200, 75, 120}	Charged (card, incl. fee): Rs 102.0 Paid (cash): Rs 50.0 Charged (card, incl. fee): Rs 204.0 Paid (cash): Rs 75.0 Charged (card, incl. fee): Rs 122.4 Total Collected: Rs 553.4

Concepts covered: instanceof, safe casting, basic inheritance (extends), for loop with a running total, if-else driven by a type check instead of a number.